Chapter 31 | Exercise solutions with Python

What Changed, and What Remains?

This notebook solves Exercises 5–8. It counts allowed causal pairs, traces when a window drops the first event, distinguishes missing from discarded information, and compares storage records with different units.

The examples run locally on short, explicit histories. The written solutions cover Exercises 1–4 and 9–12.

To run the notebook, open it inside a copy of `transformers-book` and select a Python 3 kernel. The exercises use the Python standard library. Run the cells in order to reproduce the saved outputs.

```
from pathlib import Path
import sys

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch31_perspective"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Verify a window boundary

At position 4, full causal attention can read keys 1 through 4. A window of three can read only keys 2, 3, and 4. Key 4 is the query's own position and takes one of the three slots.

Write each ordered pair as (q, k), with the query first and both positions counted from one. Full causal attention permits every $k \leq q$. The window also requires $q - k < 3$, so the key can be at the query itself or up to two positions earlier. We enumerate the pairs at each requested length and compare the sizes of the lists with the two counts from `causal_pair_counts`.

```
from examples.ch31_perspective.model import causal_pair_counts

window_width = 3
print("Length  Full causal  Windowed")
for length in (0, 3, 5):
    full_pairs = [
        (query, key)
        for query in range(1, length + 1)
        for key in range(1, query + 1)
    ]
    window_pairs = [
        (query, key) for query, key in full_pairs
        if query - key < window_width
    ]
    full_count, window_count = causal_pair_counts(length, window_width)
    assert (full_count, window_count) == (
        len(full_pairs), len(window_pairs)
    )
    print(f"{length:>6}  {full_count:>11}  {window_count:>8}")
    if length == 3:
        assert full_pairs == window_pairs
        length_three_pairs = full_pairs

print()
print("Pairs at length 3, under either rule")
print(length_three_pairs)
```

```
OUTPUT
Length  Full causal  Windowed
     0            0         0
     3            6         6
     5           15        12

Pairs at length 3, under either rule
[(1, 1), (2, 1), (2, 2), (3, 1), (3, 2), (3, 3)]
```

At length 0 there are no queries, so both counts are 0. At length 3 the queries contribute 1, 2, and 3 pairs for a total of 6. Every pair fits in the window. At length 5, full attention adds 4 pairs for query 4 and 5 pairs for query 5, giving $6 + 4 + 5 = 15$. The window drops $(4, 1)$, $(5, 1)$, and $(5, 2)$. Each of those last two queries contributes 3 pairs to the windowed count, giving $6 + 3 + 3 = 12$.

6. Change the window size

Read the code 517, then assign route X, then Y, then Z. At the end of the stream, the recent window has kept a tail of these four events. Each new event joins the tail. If the window is already full, the oldest retained event drops out.

For each width, read `first_code` and `current_route` from `recent_window`. The history slice shows which events are in the tail at the end, in the order they arrived.

```
from examples.ch31_perspective.model import compare_memory, make_history

history = make_history(517, ["X", "Y", "Z"])
expected_answers = {2: (None, "Z"), 3: (None, "Z"), 4: (517, "Z")}

print("Window  first_code  current_route  Retained events")
for width in (2, 3, 4):
    recent = compare_memory(history, width)["recent_window"]
    first_code = recent["first_code"]
    current_route = recent["current_route"]
    assert (first_code, current_route) == expected_answers[width]

    tail = history[-width:]
    assert recent["retained_units"] == len(tail)
    retained = ", ".join(f"{kind}={value}" for kind, value in tail)
    print(
        f"{width:>6}  {str(first_code):>10}  "
        f"{current_route:>13}  {retained}"
    )
```

```
OUTPUT
Window  first_code  current_route  Retained events
     2        None              Z  route=Y, route=Z
     3        None              Z  route=X, route=Y, route=Z
     4         517              Z  code=517, route=X, route=Y, route=Z
```

Width 2 keeps Y and Z. Width 3 also keeps X, but the three route assignments still fill the window and the earlier code has dropped out. In both cases `first_code` is `None`, meaning the code is unavailable from the retained events. Width 4 has room for the whole stream, so `first_code` becomes 517.

Every tail includes Z, the final route assignment. Each assignment replaces the earlier route, so that one retained event is enough to answer the current-route question at all three widths.

7. Test a history with no route assignments

The delivery starts with code 620 and receives no route assignment. The history therefore contains one event, `("code", 620)`. The full archive keeps the event. A window of one also keeps it because it is the most recent event. The code-and-route summary stores the code in its code field. The route-only summary creates a field when it sees a route assignment, so it has zero populated fields here. All four rules have no route value to return.

Read `first_code`, `current_route`, and the storage count for each method. The expected answers and counts come from tracing what each rule keeps.

```
from examples.ch31_perspective.model import compare_memory, make_history

code_only = make_history(620, [])
assert code_only == (("code", 620),)
code_only_answers = compare_memory(code_only, window=1)

expected_code_only = {
    "full_archive": (620, None, 1, "event"),
    "recent_window": (620, None, 1, "event"),
    "route_only": (None, None, 0, "field"),
    "code_and_route": (620, None, 1, "field"),
}
assert set(code_only_answers) == set(expected_code_only)
print(f"{'Rule':15} {'First code':>10} {'Route':>7} {'Count':>6} Unit")
for rule, expected in expected_code_only.items():
    answer = code_only_answers[rule]
    observed = (
        answer["first_code"],
        answer["current_route"],
        answer["retained_units"],
        answer["unit"],
    )
    assert observed == expected
    first_code, current_route, count, unit = observed
    print(
        f"{rule:15} {str(first_code):>10} {str(current_route):>7} "
        f"{count:6d} {unit}"
    )
```

```
OUTPUT
Rule            First code   Route  Count Unit
full_archive           620    None      1 event
recent_window          620    None      1 event
route_only            None    None      0 field
code_and_route         620    None      1 field
```

The output uses `None` for any unavailable answer. Here `current_route` is `None` for every rule because a route event never occurred. The route-only summary also reports `first_code=None` even though the code event occurred. That summary has no field for it. For an example of eviction, append a single route assignment to the same code and keep the window at one. The route event takes the only slot, so the window's code answer becomes `None`. The same marker covers information that was absent from the stream and information that the rule left out of its retained state.

8. Compare the storage records

With one route update the history is a code followed by route A. With eight updates it contains the code and routes A through H in order. Each sweep entry uses two different starting codes and the same routes. Changing the code changes the value of the first event while leaving the number of events and populated fields the same.

The archive keeps both events at one update and all nine at eight updates. The three-event window keeps both events in the first case and only the final three route events in the second. The route-only summary overwrites one route field on every assignment. The code-and-route summary also keeps the original code, so both its fields are populated as soon as the first route arrives.

Run the chapter's module with the notebook kernel's Python and pass `--output` as in the listing. The JSON goes to a temporary directory. Read `retained_units` and `unit` from the one-update and eight-update records in `accuracy_sweep` and check them against those counts.

```
import json
import subprocess
from tempfile import TemporaryDirectory

with TemporaryDirectory() as temp_dir:
    json_path = Path(temp_dir) / "ch31.json"
    subprocess.run(
        [
            sys.executable,
            "-m",
            "examples.ch31_perspective.run_examples",
            "--output",
            str(json_path),
        ],
        cwd=repo_root,
        check=True,
        stdout=subprocess.DEVNULL,
    )
    report = json.loads(json_path.read_text(encoding="utf-8"))

assert report["window"] == 3
methods_by_updates = {
    row["route_updates"]: row["methods"] for row in report["accuracy_sweep"]
}
```

```
expected_storage = {
    "full_archive": (2, 9, "event"),
    "recent_window": (2, 3, "event"),
    "route_only": (1, 1, "field"),
    "code_and_route": (2, 2, "field"),
}
one_update = methods_by_updates[1]
eight_updates = methods_by_updates[8]
assert set(one_update) == set(eight_updates) == set(expected_storage)

print(f"{'Rule':15} {'1 update':>8} {'8 updates':>9} Unit")
for rule, expected in expected_storage.items():
    first = one_update[rule]
    last = eight_updates[rule]
    unit = first["unit"]
    assert unit == last["unit"]
    first_count = first["retained_units"]
    last_count = last["retained_units"]
    assert (first_count, last_count, unit) == expected
    print(f"{rule:15} {first_count:8d} {last_count:9d} {unit}")
```

```
OUTPUT
Rule            1 update 8 updates Unit
full_archive           2         9 event
recent_window          2         3 event
route_only             1         1 field
code_and_route         2         2 field
```

The two event stores count entries in a history. For example, the archive keeps a separate entry for each route assignment, with the event's kind and value in a tuple. A summary counts populated fields in a dictionary. Its route field holds only the last assigned value. Two events and two fields describe equal counts of different units. A byte comparison needs a specified representation and accounting boundary. In Python, container overhead and whether keys and values are shared affect how many bytes each store occupies. These JSON records report logical counts for the stores.